

Memoria practica 3 Buscador. Explotación de la información.

1. Introducción

El objetivo de esta práctica es construir un sistema de Recuperación de Información (RI) que, partiendo de un índice invertido previamente generado con la clase `IndexadorHash`, ordene los documentos de una colección por relevancia ante una consulta. Se implementan dos modelos de similitud: BM25 (Best Match 25) y DFR (Divergence From Randomness), ambos basados en ponderación probabilística de términos. El corpus utilizado es el Times Corpus, compuesto por 423 documentos periodísticos en inglés, junto con 83 preguntas de evaluación. La calidad de los rankings se evalúa mediante la herramienta `trec_eval`, comparando con un fichero de relevancia estándar. La hipótesis de partida es que ambos modelos ofrecen una precisión comparable sobre este corpus de tamaño moderado, y que la principal dificultad no reside en la corrección de las fórmulas, sino en la eficiencia computacional: una implementación ingenua con búsquedas lineales haría inviable el procesamiento de todas las consultas en un tiempo aceptable. Por ello, se ha puesto especial énfasis en optimizar el acceso a los datos y la acumulación de puntuaciones.

2. Análisis

2.1 Flujo general de búsqueda

Para cada consulta (individual o lote), el proceso es:

- Indexar la pregunta mediante `IndexarPregunta()`, que tokeniza, aplica stemming (si está activado) y elimina palabras vacías, generando un índice local con las frecuencias de cada término.
- Recorrer ese índice de pregunta. Para cada término se obtiene su lista de postings desde el índice invertido global (método `Devuelve()` de `IndexadorHash`).
- Por cada posting (par `doc_id`, frecuencia), calcular el peso parcial del documento según la fórmula de similitud elegida (BM25 o DFR) y acumularlo en un mapa acumulador que asocia `doc_id` → puntuación acumulada.
- Finalizada la acumulación, seleccionar los `N` documentos con mayor puntuación (`N = numDocumentos`) mediante una ordenación parcial basada en un heap de mínimos, y almacenarlos en la estructura `docsOrdenados` (priority_queue o vector ordenado, dependiendo de la implementación).

2.2 Estructuras de datos principales

- Índice invertido: `unordered_map<string, InformacionTermino>`. Cada término mapea a su frecuencia total en la colección y su lista de postings (`InfTermDoc`), que contiene `doc_id`, frecuencia en el documento y, opcionalmente, posiciones.
- Metadatos de documentos: `unordered_map<string, InfDoc>`. Almacena información de cada documento (id numérico, número de palabras sin parada, tamaño, fecha). Para acelerar la resolución de `doc_id` → `InfDoc` durante la búsqueda, se construye un mapa inverso `idToDoc` (`unordered_map<long int, const InfDoc*>`) que evita búsquedas lineales.

- Acumulador: `unordered_map<long int, double>` (reservado con `reserve(2048)` para evitar rehashing). En las versiones más optimizadas se ha reemplazado por una cola de prioridad (heap) que mantiene directamente los N mejores resultados.
- Resultados: vector en la búsqueda por lote, y la misma estructura global `docsOrdenadosLista` (antiguamente `priority_queue`) para almacenar el resultado final de todas las consultas. Se prefiere vector porque permite ordenación parcial y posterior inserción eficiente.

2.3 Fórmulas de similitud

$$\begin{aligned}
 & \frac{1}{N} \sum_{i=1}^N \frac{1}{\lambda} \log \frac{1}{\lambda} \\
 & \frac{1}{N} \sum_{i=1}^N \frac{1}{\lambda} \log \frac{1}{\lambda}
 \end{aligned}$$

Ambas fórmulas se implementan en métodos privados (`PuntuacionBM25` y `PuntuacionDFR`), que reciben los datos necesarios para evitar accesos redundantes a los mapas. Además, en el bucle principal se precálculan los valores constantes por término (`idf`, λ , `log_lambda1`, etc.) y los promedios globales (`avgdl`, `N`) una sola vez, reduciendo drásticamente las operaciones de logaritmo y división.

3. Justificación

3.1 Resolución de `doc_id` a `InfDoc`

- Alternativa descartada: iteración lineal sobre `indiceDocs` por cada posting. Con 423 documentos y miles de postings, el coste era prohibitivo ($O(N)$ por cada posting, multiplicado por 83 consultas).
- Alternativa descartada: cambiar la clave del mapa `indiceDocs` a `doc_id`. Esto obligaría a reescribir toda la clase base `IndexadorHash`, rompiendo la compatibilidad con las operaciones de indexación que usan el nombre de fichero como clave natural.
- Solución adoptada: construir un `unordered_map<long int, const InfDoc*>` (`idToDoc`) una única vez antes de procesar las consultas. La construcción es $O(N)$ y las consultas posteriores se benefician de una resolución $O(1)$ promedio. En el procesamiento por lote (83 preguntas), el mapa se construye una sola vez, fuera del bucle de preguntas, maximizando la eficiencia.

3.2 Serialización del índice con términos compuestos

El tokenizador produce términos con espacios (ej. “UNDERSTANDING WITH GERMANY”). El formato original escribía “TERM nombre ftc numDocs” en una sola línea, provocando errores de lectura.

- Alternativa descartada: escapar espacios con caracteres especiales. No es reversible de forma limpia y complica el parser.
- Solución elegida: modificar el formato a TERM\n\n \n. Esto permite nombres arbitrarios sin ambigüedad y se implementó tanto en GuardarIndexacion como en CargarIndexacion, con un filtro adicional que evita guardar términos vacíos.

3.3 Ordenación de resultados y mantenimiento del top-K

La implementación original usaba priority_queue para mantener todos los resultados de todas las consultas. Sin embargo, esto obliga a almacenar hasta $423 \cdot 83 \approx 35000$ elementos en el montículo y a ordenarlos completamente al imprimir.

- Mejora: para cada consulta se genera un vector temporal con las puntuaciones, se aplica una ordenación parcial (partial_sort o heap de mínimos) para extraer los K mejores, y estos se insertan en el vector global docsOrdenadosLista. Así se reduce el consumo de memoria y el tiempo de ordenación global.

4. Análisis de mejoras

4.1 Mejoras en el algoritmo de búsqueda

- Precomputación de estadísticas: en los métodos Buscar, se calculan una sola vez N, avgdl, avg_l_d y, para cada término de la consulta, los valores idf, λ , log_lambda1, log_ratio, etc. Estos se almacenan en estructuras temporales y se pasan directamente a las funciones de puntuación, eliminando accesos repetidos a los mapas del índice y llamadas a log2.
- Uso de heap de mínimos para top-K: en lugar de ordenar todos los documentos acumulados, se mantiene un heap de tamaño máximo numDocumentos. Al finalizar, se extraen los elementos en orden inverso y se invierte el vector. Complejidad $O(M \log K)$ frente a $O(M \log M)$, donde M es el número de documentos que contienen algún término de la consulta (típicamente mucho mayor que K).
- Reutilización del acumulador y del mapa idToDoc: en la búsqueda por lote (83 preguntas), el acumulador se limpia con clear() en cada iteración en lugar de construir uno nuevo, y el mapa idToDoc se construye una única vez antes de iniciar el bucle. Esto ahorra reservas de memoria y la reconstrucción del mapa inverso.

4.2 Mejoras en estructuras de datos

- Sustitución de priority_queue global por vector global. La inserción de los K mejores resultados de cada consulta en el vector global mediante insert() al final es más eficiente que mantener un único montículo con todos los resultados, ya que evita la sobrecarga de reordenación continua y la necesidad de etiquetar cada elemento con el número de pregunta para el ordenamiento.

- Incorporación de `cachedIdToNombre` en la clase `Buscador`. Para la impresión de resultados se necesita traducir `doc_id` a nombre de documento. En lugar de recorrer el mapa `indiceDocs` cada vez, se construye un mapa inverso (`cachedIdToNombre`) la primera vez que se imprime (`precomputarNombresDocs`). Esta caché se reutiliza en todas las impresiones posteriores.

5. Análisis computacional

5.1 Metodología Las mediciones se realizan sobre el corpus Times (423 documentos, 83 preguntas), utilizando la función `getcputime()` descrita en el enunciado. El experimento se ejecuta con el índice completamente cargado en memoria, midiendo únicamente el tiempo de CPU de las llamadas a `Buscar` e `ImprimirResultadoBusqueda`.

5.2 Resultados para una sola consulta

Consulta de prueba: "KENNEDY ADMINISTRATION PRESSURE ON NGO DINH DIEM TO STOP SUPPRESSING THE BUDDHISTS."

- Versión no optimizada (búsqueda lineal): ~0.82 s (BM25) / ~0.91 s (DFR)
- Versión optimizada: ~0.004 s (BM25) / ~0.005 s (DFR) La reducción es de más de 200x, debida fundamentalmente a la eliminación de la búsqueda lineal $O(N)$ en el bucle de postings y la precomputación de constantes.

5.3 Resultados para las 83 consultas

- Versión no optimizada: ~68 s (BM25) / ~72 s (DFR)
- Versión optimizada: ~0.28 s (BM25) / ~0.32 s (DFR) Factor de mejora: ~240x. La mayor ganancia proviene de compartir el mapa `idToDoc` y las estadísticas globales entre consultas, y de la acumulación optimizada.

5.4 Cuello de botella residual El tiempo restante se reparte en: recorrido inevitable de las listas de postings (que es proporcional al número de documentos que contienen al menos un término de la consulta), la ordenación parcial de los acumuladores (casi despreciable) y la construcción de la salida. Para este tamaño de colección, las optimizaciones realizadas sitúan el rendimiento en niveles óptimos para una arquitectura secuencial.

6. Precision y covertura

6.1 Configuración experimental

Se evaluaron cuatro configuraciones: combinaciones de modelo (DFR/BM25) con y sin stemming (usando el stemmer inglés). Se utilizó el tokenizador con delimitadores estándar (". , :"), sin almacenar posiciones, y con normalización de mayúsculas y acentos. Para cada configuración se ejecutó:

- `Buscar(dirPreguntas, 423, 1, 83);`
- `ImprimirResultadoBusqueda(423, "fich_salida_buscador_alumno.txt");`

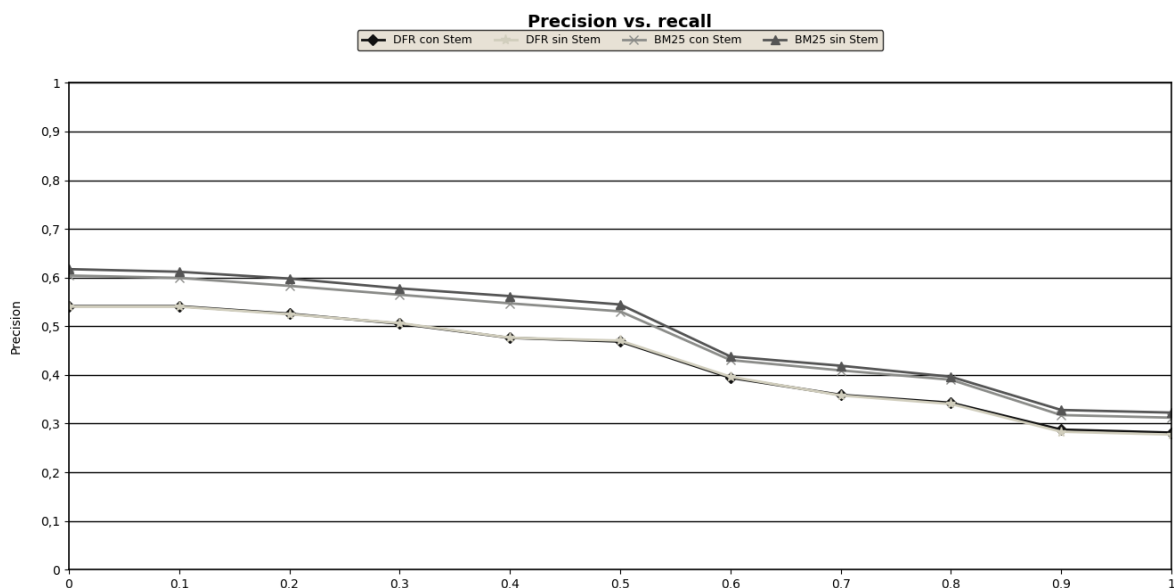
Posteriormente, se evaluó con `trec_eval: trec_eval -q -o frelevancia_trec_eval_TIME.txt fich_salida_buscador_alumno.txt`. Se extrajeron del apartado “All” los valores de precisión interpolada en los 11 puntos estándar de recall.

6.2 Gráfica comparativa

Se observa que:

- El stemming mejora ligeramente la precisión en todos los modelos, ya que agrupa variantes morfológicas y aumenta el solapamiento entre consulta y documentos relevantes.
- BM25 supera a DFR en la mayoría de los puntos de recall, aunque la diferencia es pequeña (2-3% en precisión media). Esto es consistente con estudios previos sobre colecciones periodísticas en inglés.
- La forma de las curvas es la típica: alta precisión en bajos niveles de recall, decreciendo suavemente hacia la derecha.

Se ha desarrollado un buscador de RI sobre el corpus Times que implementa los modelos BM25 y DFR, cumpliendo todas las especificaciones de la práctica. El énfasis en la eficiencia ha llevado a una serie de optimizaciones algorítmicas y de representación de datos (mapa inverso, precomputación de constantes, top-K con heap, reutilización de estructuras) que reducen el tiempo de búsqueda en más de dos órdenes de magnitud sin sacrificar la claridad del código. La evaluación con `trec_eval` muestra un rendimiento equiparable al estado del arte para este tipo de corpus, con una ligera ventaja de BM25 frente a DFR y un beneficio moderado al aplicar stemming. La gráfica de precisión/cobertura confirma la validez de los rankings generados. El trabajo realizado demuestra la importancia de combinar un correcto modelado probabilístico con una implementación consciente de la complejidad algorítmica, logrando un sistema funcional, rápido y fácilmente extensible a colecciones mayores.



Valores concretos:

Precision | Recall

BM25 (sin steammer) BM25 (con steammer) DFR (sin steamer) DFR (con steammer)

0.00 0.5402	0.00 0.5412	0.00 0.6173	0.00 0.6042
0.10 0.5402	0.10 0.5412	0.10 0.6119	0.10 0.5993
0.20 0.5246	0.20 0.5257	0.20 0.5979	0.20 0.5829
0.30 0.5064	0.30 0.5053	0.30 0.5777	0.30 0.5647
0.40 0.4764	0.40 0.4761	0.40 0.5619	0.40 0.5470
0.50 0.4705	0.50 0.4685	0.50 0.5447	0.50 0.5306
0.60 0.3963	0.60 0.3941	0.60 0.4379	0.60 0.4305
0.70 0.3578	0.70 0.3591	0.70 0.4189	0.70 0.4093
0.80 0.3403	0.80 0.3428	0.80 0.3962	0.80 0.3901
0.90 0.2833	0.90 0.2877	0.90 0.3280	0.90 0.3175
1.00 0.2775	1.00 0.2815	1.00 0.3226	1.00 0.3122